

Recomendadores Generativos I: Recuperación de Información Generativa

IIC3633 – Sistemas Recomendadores

Denis Parra

Profesor Asociado

PUC Chile

Contenidos

1. Motivación: retrieval con atomic item IDs
2. Semantic IDs: la idea clave y su intuición
3. Construcción de Semantic IDs con RQ-VAE
4. Generative retrieval con un Transformer
5. Resultados, ablaciones y nuevas capacidades
6. TIGER vs. VQ-Rec (Hou et al., WWW 2023)
7. Limitaciones, trabajos posteriores y mensajes clave

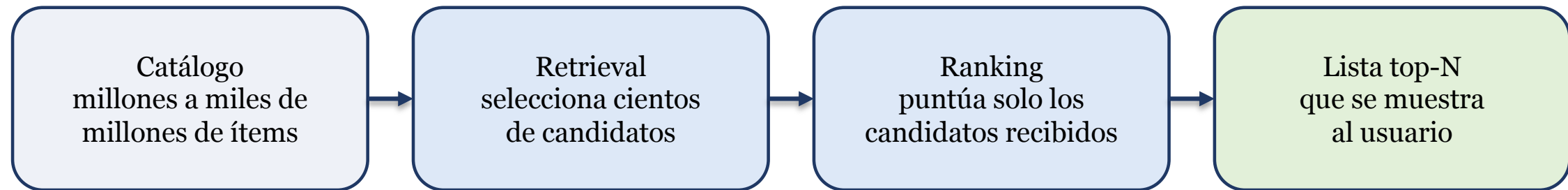
Mensajes clave

- **Semantic IDs** = tuplas cortas de codewords derivadas del contenido del ítem: ítems similares comparten prefijos
- **RQ-VAE** los construye mediante residual quantization; un 4.º token resuelve las colisiones
- **TIGER** trata la recomendación como generación de secuencias: decodifica el Semantic ID del siguiente ítem
 - Beneficios: mejor precisión, ítems cold-start, control de la diversidad
- **VQ-Rec** también cuantiza texto, pero para indexar embeddings con fines de transferencia, no para generar

Por qué el retrieval necesita una nueva idea

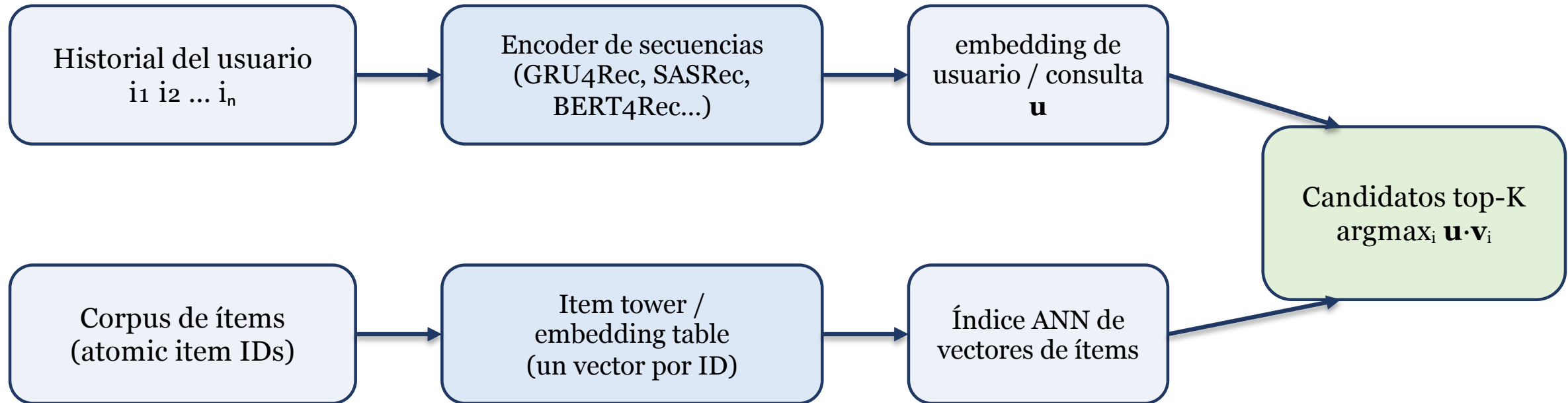
MOTIVACIÓN

Recomendación a gran escala: retrieval y luego ranking



- El ranker solo trabaja sobre lo que entrega el retrieval, por lo que este debe devolver candidatos **muy relevantes** (alto recall)
- El retrieval debe ser rápido: no puede puntuar cada ítem de un catálogo enorme con un modelo pesado
- Solución estándar: representar consultas e ítems en el **mismo espacio** de embeddings y usar búsqueda de **vecinos más cercanos aproximados (ANN)**

Retrieval estándar: embeddings + búsqueda ANN



La factorización matricial, los dual encoders two-tower y sus variantes secuenciales siguen esta receta: codificar y luego buscar en un **índice separado y pre-contruido** mediante maximum inner product search (MIPS).

Límites de los atomic IDs + embedding tables

- Cada ítem es un **atomic ID** aleatorio; su embedding se aprende solo a partir de las interacciones de ese mismo ítem
- No se comparte conocimiento entre ítems similares: problema para el **cold-start** y la cola larga (long tail)
- La embedding table crece **linealmente** con el catálogo (miles de millones de ítems → alto consumo de memoria)
- Feedback loop: los ítems que ya tienen datos se recuperan más y, por lo tanto, acumulan aún más datos
- Se debe construir, almacenar y mantener actualizado un índice ANN separado

Una idea distinta: generative retrieval

- En lugar de emparejar consulta y candidato, un **modelo generativo end-to-end** predice directamente el ID del candidato
- La memoria del Transformer (sus parámetros) actúa como **índice**, como en DSI para document retrieval (Tay et al. 2022)
- **TIGER** = Transformer Index for GEnenerative Recommenders (Rajput et al., Google, NeurIPS 2023)
- Pregunta: ¿qué ID puede *generar* un modelo, token por token? → **Semantic IDs**

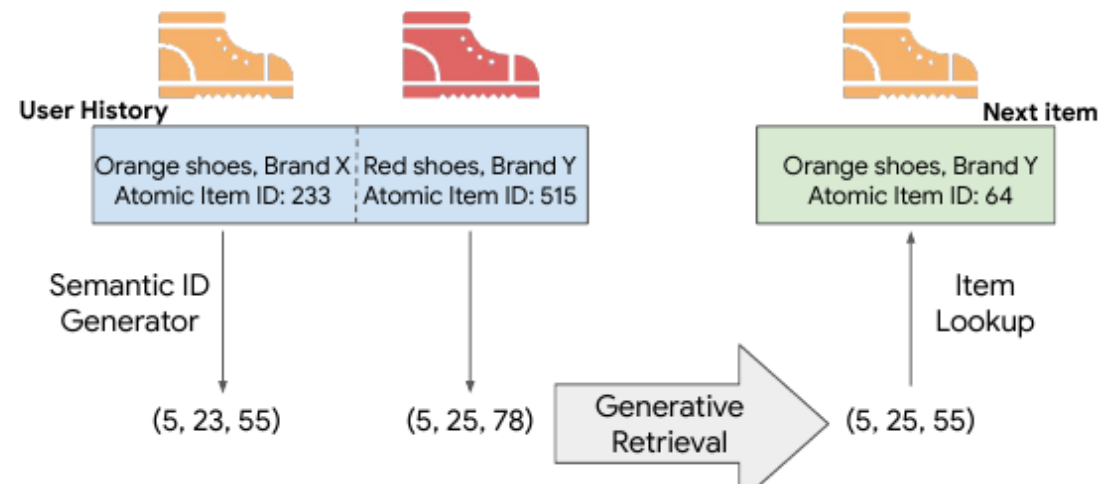


Figura 1 del artículo: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.

Contribuciones del artículo

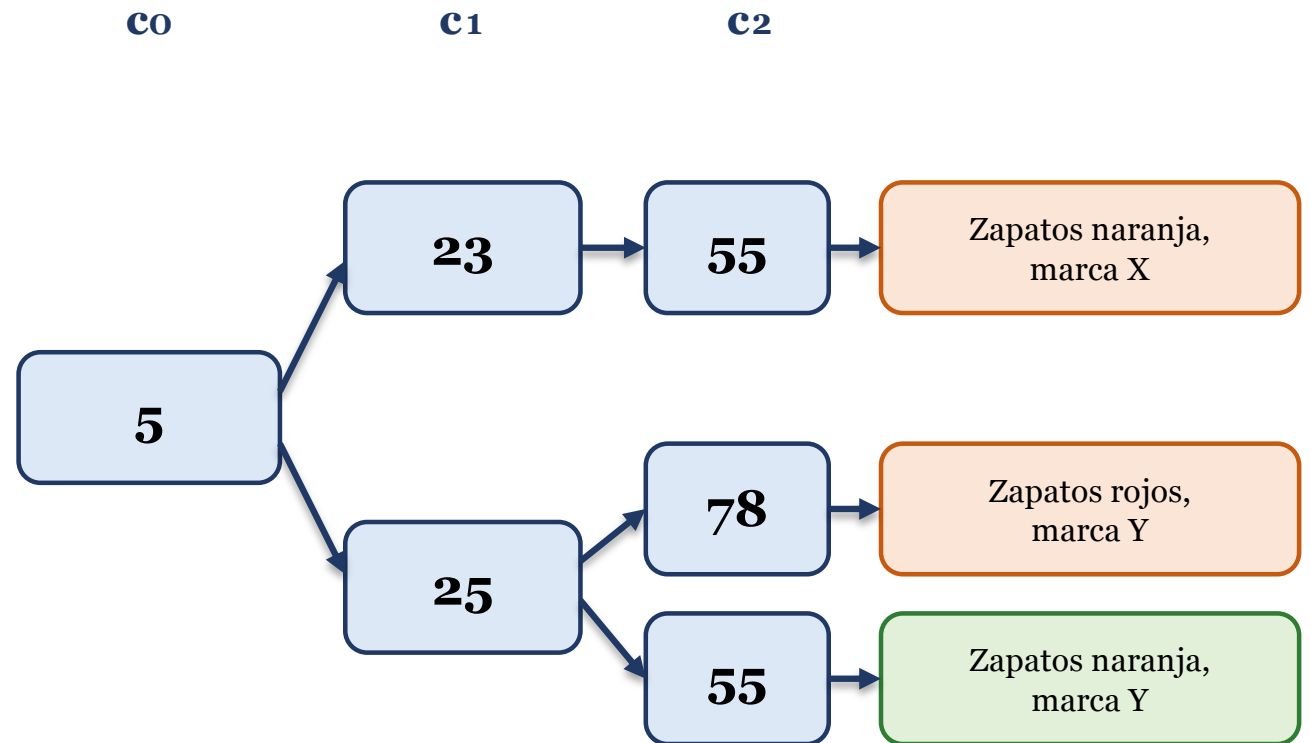
- **TIGER:** un framework de generative retrieval: asignar a cada ítem un **Semantic ID** y entrenar un modelo que prediga el Semantic ID del siguiente ítem
- **Desempeño:** supera a recomendadores secuenciales del estado del arte en tres datasets de Amazon (Recall@K, NDCG@K)
- **Nuevas capacidades:** mejor cold-start (ítems no vistos o poco frecuentes) y diversidad ajustable de las recomendaciones
- Sin índice ANN que mantener: la embedding table **no** crece linealmente con el corpus de ítems

La idea clave y su intuición

SEMANTIC IDS

Semantic ID: un ítem como tupla de codewords

- Una **tupla ordenada de m codewords**, cada uno de un codebook distinto, derivada del **contenido** del ítem
- **Coarse** → **fine**: el primer codeword = categoría amplia; los siguientes la refinan
- **Ítems similares comparten prefijos**: (10, 21, 35) está más cerca de (10, 21, 40) que de (10, 23, 32)
- Identidad de un ítem = un **camino único en un árbol**
- Capacidad = producto de los tamaños de los codebooks



Ejemplo de la Figura 1 del artículo (el ítem verde es el siguiente ítem a predecir)

Intuición: direcciones en lugar de números de serie

Atomic item IDs

233 515 64

- Una etiqueta aleatoria: el ID 233 no está más “cerca” de 234 que de 9.000.000
- El modelo debe aprender cada ítem desde cero

Semantic IDs

(5, 23, 55) (5, 25, 78) (5, 25, 55)

- Ítems con contenido similar comparten los primeros codewords
- La fuerza estadística se comparte entre ítems similares

Como una dirección postal: país → ciudad → calle → número

c0 categoría amplia
(p. ej., cabello, piel)

c1 subcategoría
dentro de c0

c2 grupo específico
de productos

c3 desempate en caso
de colisiones

Ejemplo: Semantic Ids y tries de videos de deportes

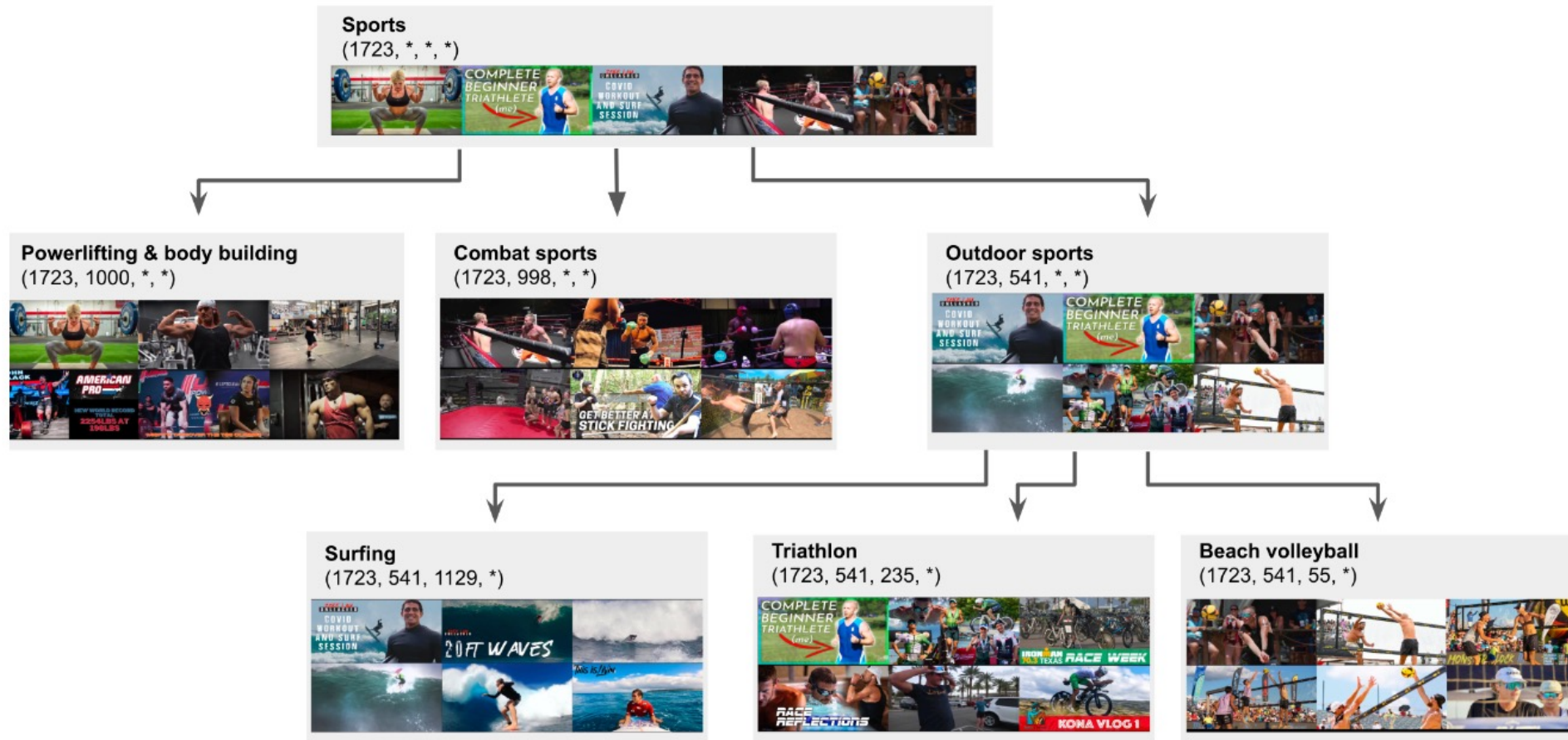


Figure 6: A sub-trie that exhibits hierarchical structures with Semantic IDs capturing sports videos.

Por qué ayudan los Semantic IDs

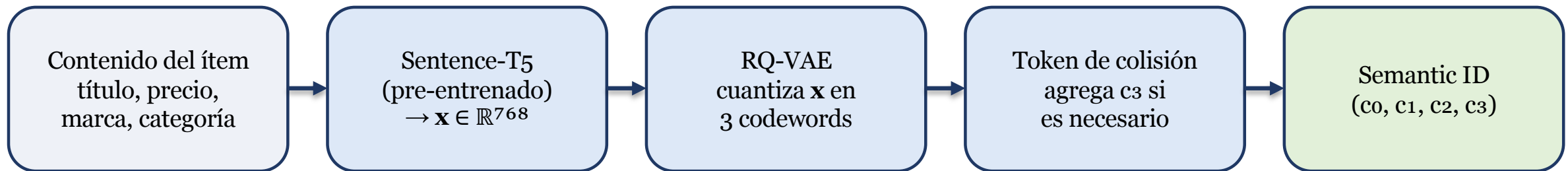
- **Conocimiento compartido:** ítems similares comparten tokens, de modo que los parámetros del modelo se entrenan conjuntamente con ítems relacionados
- **Generalización / cold-start:** un ítem nuevo obtiene su Semantic ID solo a partir de su contenido, sin necesidad de historial de interacciones
- **Compacidad:** m tokens de K codewords pueden direccionar K^m ítems; el artículo usa $4 \times 256 = \mathbf{1.024 \text{ item tokens}}$ para catálogos de 10K–20K ítems (hasta ~4.300 millones de IDs posibles)
- **Menor sensibilidad al feedback loop:** el modelo no depende de atomic IDs aleatorios como atributos de los ítems

Content embedding $\rightarrow$ RQ-VAE $\rightarrow$ codewords

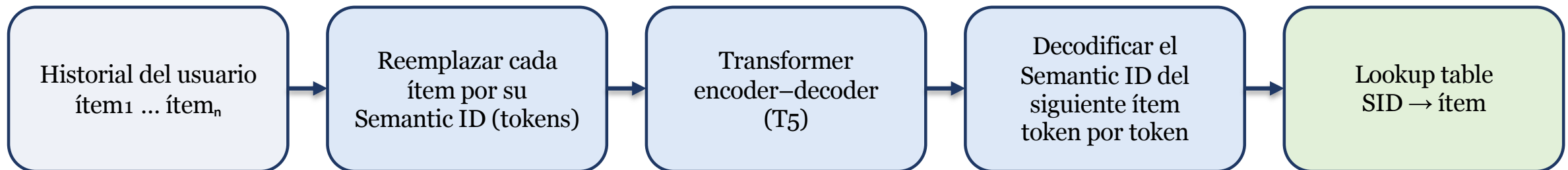
CONSTRUCCIÓN DE SEMANTIC IDS

TIGER en dos etapas

Etapa 1 – Generación de Semantic IDs



Etapa 2 – Generative retrieval sobre Semantic IDs

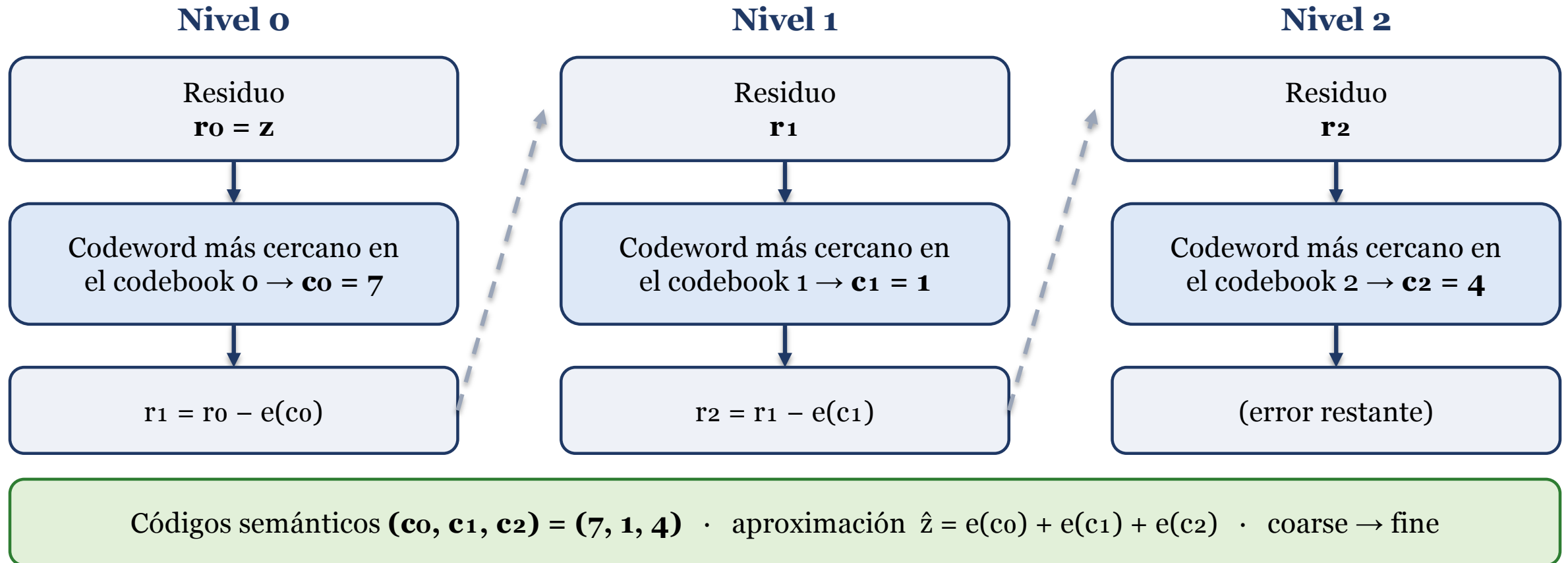


Las dos etapas se entrenan por separado: primero se entrena el cuantizador y luego se deja fijo.

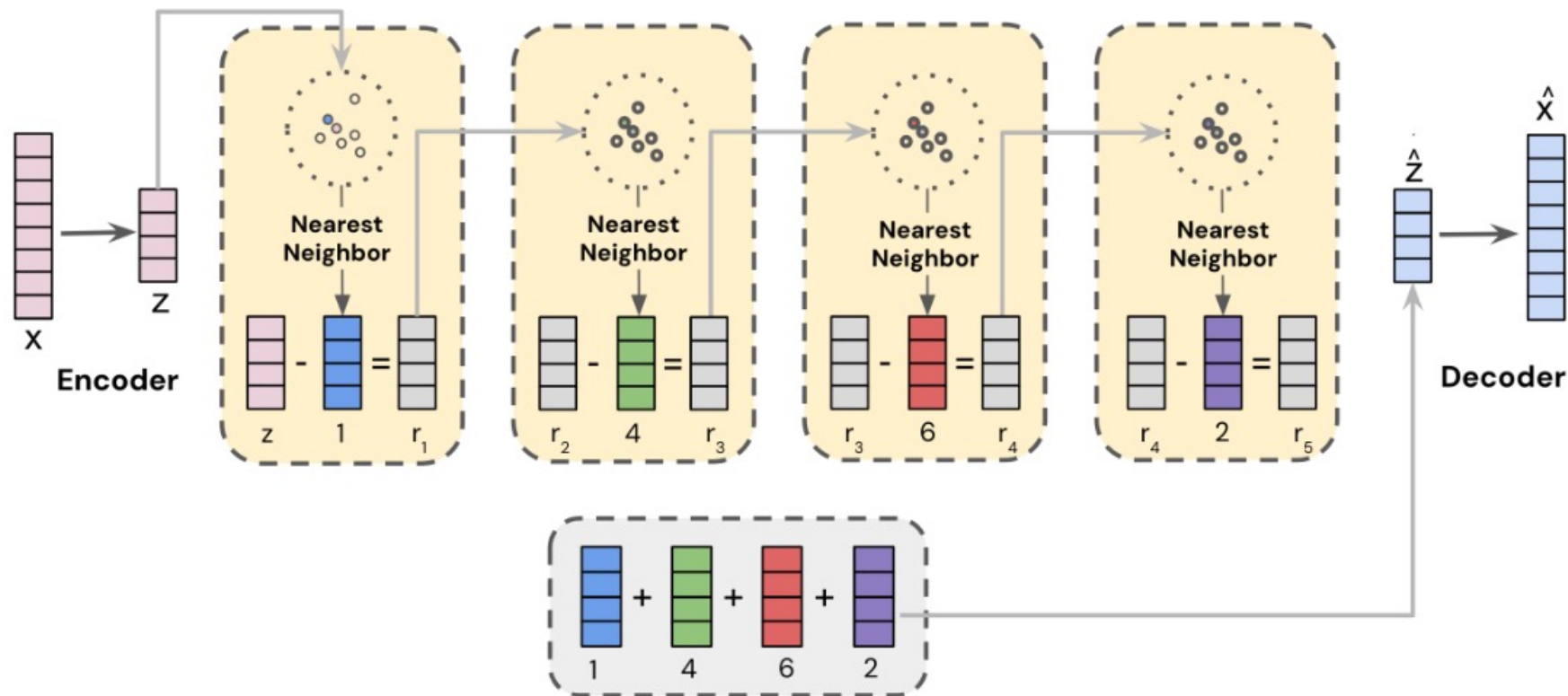
Paso 1: content embeddings

- Cada ítem tiene **atributos de contenido**: título, descripción, precio, marca, categoría (también se podrían usar imágenes)
- TIGER concatena los campos en una sola oración y la entrega a un text encoder pre-entrenado: **Sentence-T5** $\rightarrow$ embedding $\mathbf{x} \in \mathbb{R}^{768}$
- Se pueden usar otros encoders (p. ej., BERT): el cuantizador solo necesita un embedding semántico
- **Propiedad deseada**: ítems con contenido similar (embeddings cercanos) deberían recibir Semantic IDs que se **superponen**

Residual quantization: la intuición



RQ-VAE: encoder $\rightarrow$ residual quantizer $\rightarrow$ decoder



La salida r_0 del encoder (DNN) se cuantiza nivel por nivel: codeword más cercano $\rightarrow$ residuo $\rightarrow$ siguiente nivel. La suma de los codewords elegidos pasa a un decoder que reconstruye el embedding de entrada.

RQ-VAE: codificación y residual quantization

Se codifica el content embedding $\mathbf{x}$ con el encoder E ; el primer residuo es el vector latente

$$\mathbf{z} = E(\mathbf{x}), \quad \mathbf{r}_0 = \mathbf{z}$$

En el nivel d : se elige el **codeword más cercano** del codebook $C^d = \{\mathbf{e}_k\}$, $k = 1..K$
→ codeword c_d

$$c_d = \arg \min_k \|\mathbf{r}_d - \mathbf{e}_k^{(d)}\|$$

El **residuo** pasa al siguiente nivel; m niveles entregan el Semantic ID ($c_0, \dots, c_{m-1}$)

$$\mathbf{r}_{d+1} = \mathbf{r}_d - \mathbf{e}_{c_d}^{(d)}, \quad d = 0, \dots, m-1$$

El latente cuantizado es la **suma** de los codewords elegidos; el decoder D intenta recuperar $\mathbf{x}$

$$\hat{\mathbf{z}} = \sum_{d=0}^{m-1} \mathbf{e}_{c_d}^{(d)}, \quad \hat{\mathbf{x}} = D(\hat{\mathbf{z}})$$

RQ-VAE: objetivo de entrenamiento

Loss total = reconstrucción + términos de cuantización

$$\mathcal{L}(\mathbf{x}) = \mathcal{L}_{recon} + \mathcal{L}_{rqvae}$$

Reconstrucción: la salida del decoder debe coincidir con el embedding de entrada

$$\mathcal{L}_{recon} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

Cuantización: acerca codewords y residuos; compromete al encoder con los codewords (**sg** = stop-gradient, **β** = **0.25**)

$$\mathcal{L}_{rqvae} = \sum_{d=0}^{m-1} \|\text{sg}[\mathbf{r}_d] - \mathbf{e}_{c_d}\|^2 + \beta \|\mathbf{r}_d - \text{sg}[\mathbf{e}_{c_d}]\|^2$$

RQ-VAE en TIGER: configuración

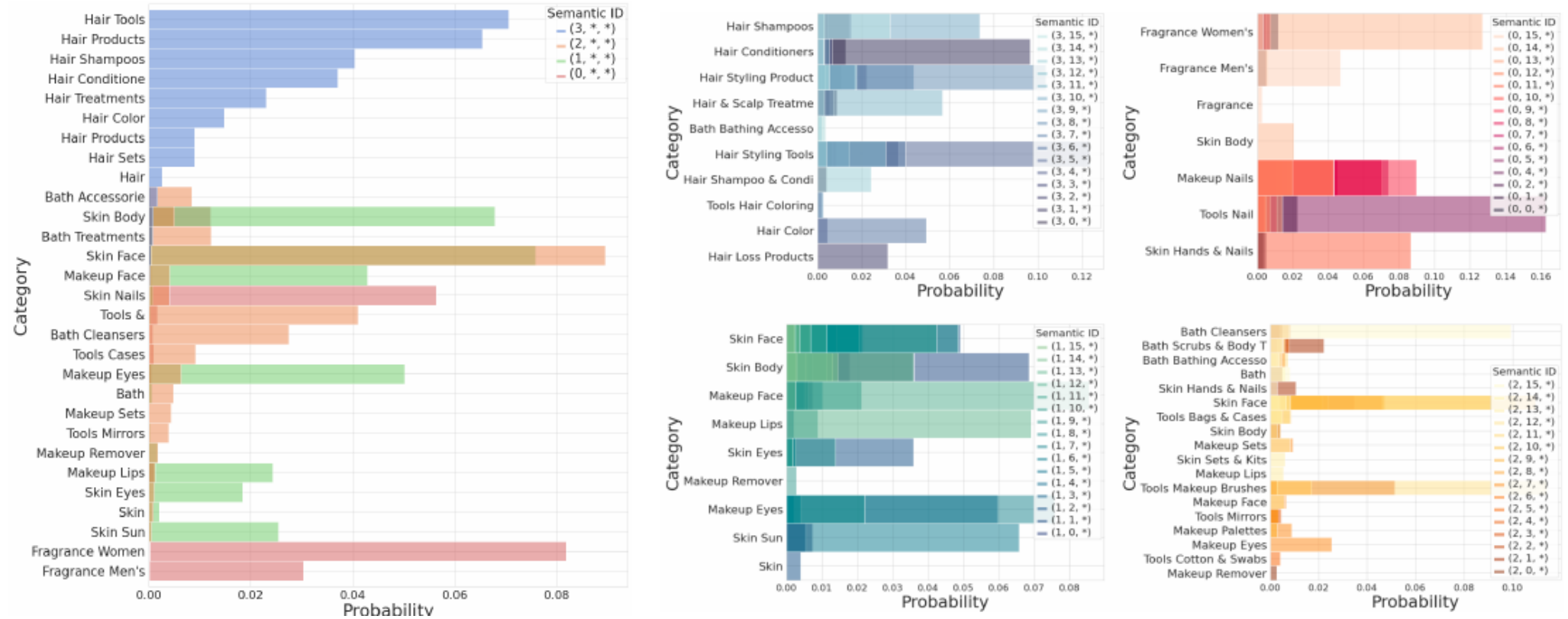
Componente	Configuración
Content encoder	Sentence-T5 → embedding de 768 dimensiones
Encoder (MLP)	3 capas ocultas: 512 → 256 → 128, ReLU; dimensión latente 32
Residual quantizer	3 niveles; un codebook por nivel: 256 vectores de dimensión 32
Loss	$L = L_{\text{recon}} + L_{\text{rqvae}}$, $\beta = 0.25$; inicialización de los codebooks con k-means
Optimización	Adagrad, learning rate 0.4, batch size 1024, 20k épocas (uso del codebook $\geq 80\%$)
Salida	3 codewords + 1 token de colisión = 4 tokens → vocabulario de $4 \times 256 = 1.024$

Fuente: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023. §4 (detalles de implementación del RQ-VAE)

Manejo de colisiones

- Ítems distintos pueden quedar asignados a los mismos codewords, p. ej. (12, 24, 52)
- Solución: agregar un **token extra** para que el ID sea único: (12, 24, 52, **0**) y (12, 24, 52, **1**)
- Una lookup table Semantic ID $\rightarrow$ ítem(s) detecta las colisiones; esto se hace **una sola vez**, después de entrenar el RQ-VAE
- Si no hay colisión, el 4.º token es simplemente 0 $\rightarrow$ todo ítem tiene un Semantic ID de largo 4
- La lookup table es barata: los IDs son tuplas de enteros, no embeddings de alta dimensión

¿Tienen significado los codewords? Amazon Beauty



c_0 captura la categoría general (p. ej., $c_0 = 3 \approx$ “Hair”; $c_0 = 1 \approx$ “Makeup” y “Skin”); con c_0 fijo, el segundo codeword c_1 separa categorías más específicas.

Figura 4 del artículo: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.

Alternativas a RQ-VAE

- **LSH:** hiperplanos aleatorios w_i , $h = 8$ bits por codeword, repetido $m = 4$ veces para lograr una cardinalidad comparable
- **K-means jerárquico:** funciona, pero pierde la relación semántica entre clusters
- **VQ-VAE:** calidad de retrieval similar, pero un código plano pierde la **jerarquía** → menos capacidades nuevas
- **IDs aleatorios:** m codewords aleatorios por ítem; el baseline sin semántica

$$c_0 = \sum_{i=1}^h 2^{i-1} \mathbf{1}[\mathbf{w}_i^\top \mathbf{x} > 0]$$

Un Transformer que decodifica Semantic IDs

GENERATIVE RETRIEVAL

La tarea: predecir el Semantic ID del siguiente ítem

Se ordenan cronológicamente los ítems de cada usuario y se **reemplaza cada ítem por sus m codewords** (entrada del modelo)

$$(c_{1,0}, \dots, c_{1,m-1}, c_{2,0}, \dots, c_{2,m-1}, \dots, c_{n,0}, \dots, c_{n,m-1})$$

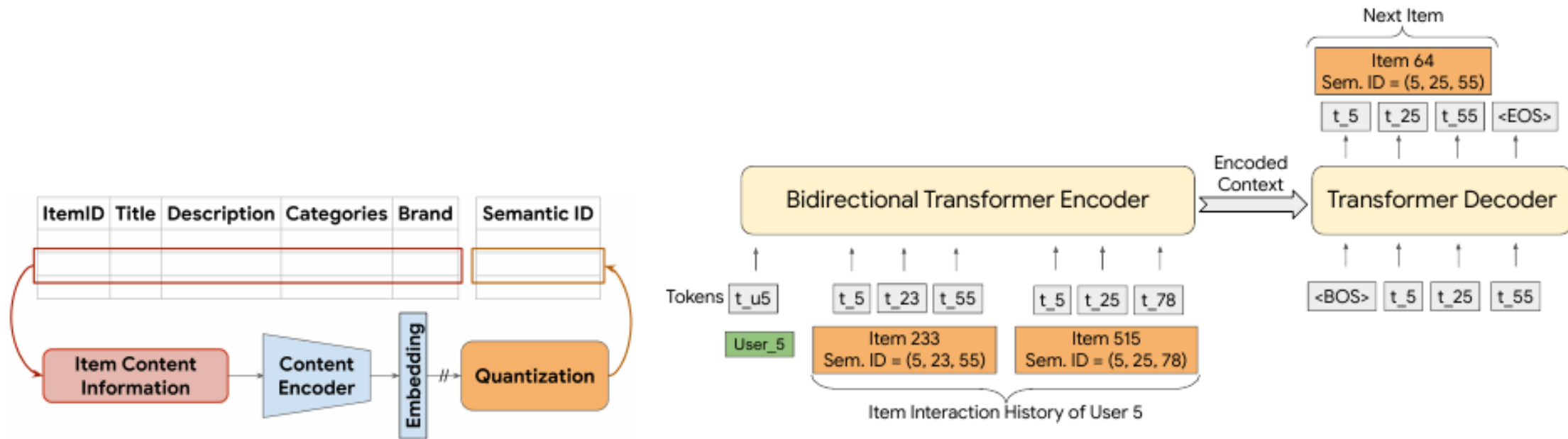
El modelo debe producir el **Semantic ID del ítem n+1** (el objetivo)

$$(c_{n+1,0}, \dots, c_{n+1,m-1})$$

La decodificación seq2seq se factoriza de forma autorregresiva: un codeword a la vez, dado el historial y los codewords ya generados

$$p(c_{n+1,0:m-1} \mid \text{history}) = \prod_{t=0}^{m-1} p(c_{n+1,t} \mid c_{n+1,<t}, \text{history})$$

TIGER: generación de Semantic IDs + encoder–decoder



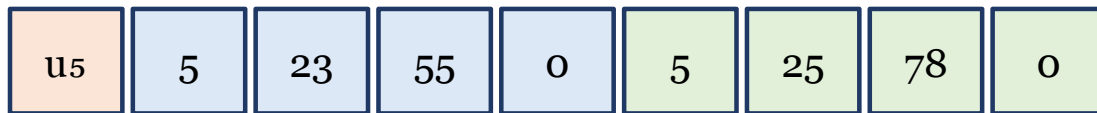
(a) Semantic ID generation for items using quantization of content embeddings.

(b) Transformer based encoder-decoder setup for building the sequence-to-sequence model used for generative retrieval.

(a) Se cuantizan los content embeddings en Semantic IDs. (b) Un encoder Transformer lee el token de usuario y los Semantic IDs del historial; el decoder genera el Semantic ID del siguiente ítem.

Entradas y salidas, token por token

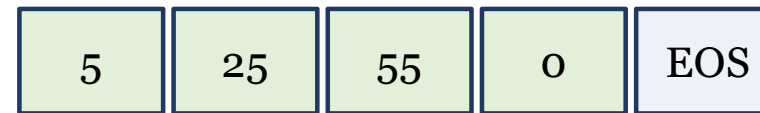
Encoder: token de usuario + historial



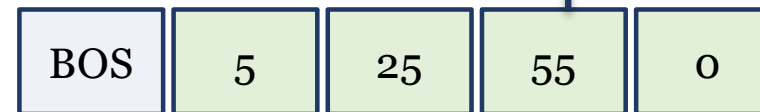
Encoder Transformer bidireccional
(4 capas, 6 heads)

- Tokens de usuario: el ID crudo se mapea con hashing a **2.000 user tokens**
- Item tokens: un vocabulario por nivel, $4 \times 256 = \mathbf{1.024}$
- Salida: 4 codewords y luego EOS → lookup del ítem

Salida del decoder (siguiente ítem)



Decoder Transformer
(4 capas, 6 heads; cross-attention)



Entrada del decoder (desplazada a la derecha)

Modelo secuencial: detalles de implementación

Aspecto	Configuración
Vocabulario	1.024 item tokens (4×256) + 2.000 tokens de ID de usuario (hashing trick)
Secuencia de entrada	token de usuario seguido de los tokens de Semantic ID del historial del usuario
Arquitectura	encoder–decoder estilo T5, 4 + 4 capas, 6 heads de dimensión 64, ReLU
Dimensiones	dimensión del modelo 128, dimensión MLP 1024, dropout 0.1 $\rightarrow \approx 13\text{M}$ parámetros
Entrenamiento	200k pasos (Beauty, Sports), 100k (Toys); batch size 256; implementado en T5X
Learning rate	0.01 durante los primeros 10k pasos y luego decaimiento inverso a la raíz cuadrada

Fuente: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023. §4 (detalles de implementación del modelo sequence-to-sequence)

Inferencia: de los tokens de vuelta a los ítems

- Se decodifican autorregresivamente los m codewords del siguiente ítem (beam search) → cada Semantic ID se mapea a un ítem con la lookup table
- El modelo puede generar un **ID inválido** (que ningún ítem tiene): $256^4 \approx 4.300$ millones de IDs posibles vs. solo 10–20K ítems
- Aun así, en las predicciones top-10 solo $\approx 0.1\%–1.6\%$ son inválidas; solución: un beam más amplio y filtrado
- Extensión sugerida por el artículo: **prefix matching** (recuperar ítems que comparten un prefijo = predecir una categoría)
- Trabajos posteriores restringen la decodificación a IDs válidos con un **trie** (constrained beam search)

Desempeño, ablaciones y nuevas capacidades

RESULTADOS

Recomendación secuencial: TIGER vs. baselines

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	<u>0.0249</u>	0.0605	0.0318	<u>0.0463</u>	<u>0.0306</u>	0.0675	0.0374
S ³ -Rec [44]	<u>0.0251</u>	<u>0.0161</u>	<u>0.0385</u>	<u>0.0204</u>	<u>0.0387</u>	0.0244	<u>0.0647</u>	<u>0.0327</u>	0.0443	0.0294	<u>0.0700</u>	<u>0.0376</u>
TIGER [Ours]	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

Amazon Beauty, Sports y Toys. Ejemplo (Beauty): NDCG@5 de 0.0321 para TIGER vs. 0.0249 para SASRec, el baseline más fuerte en ese dataset.

Ablación: ¿cómo deberían construirse los IDs?

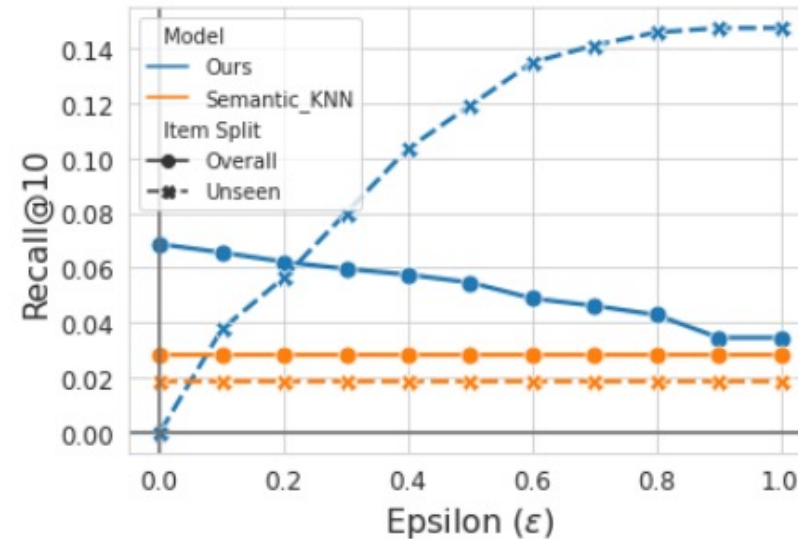
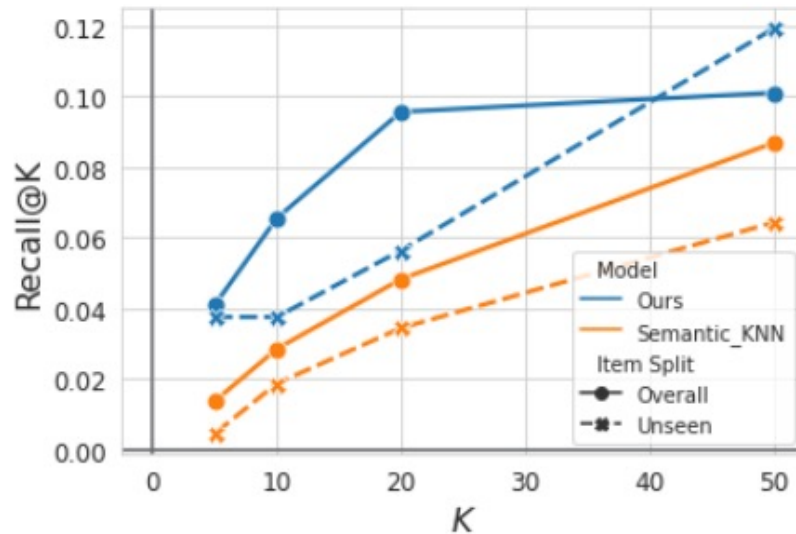
Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
Random ID	0.007	0.005	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH SID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE SID	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432

Random IDs < LSH Semantic IDs < RQ-VAE Semantic IDs: lo que ayuda es la estructura jerárquica basada en el contenido.

Tabla 2 del artículo: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.

Nueva capacidad 1: ítems cold-start

- Ítems no vistos en el entrenamiento igual obtienen un Semantic ID desde su contenido
- Candidatos = ítems con el prefijo predicho (c_0, c_1, c_2), incluidos ítems no vistos
- ε : proporción máxima de ítems no vistos en el top-K
- TIGER supera a Semantic_KNN (vecinos en el espacio de content embeddings) para $\varepsilon \geq 0.1$



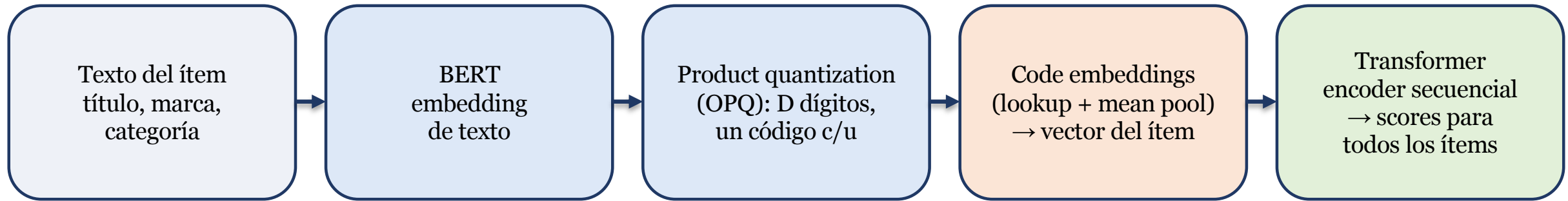
Nueva capacidad 2: diversidad controlable

- Muestrear con temperatura durante la decodificación cambia qué Semantic IDs se generan
- Mayor temperatura $\rightarrow$ predicciones repartidas en más categorías (mayor Entropy@K)
- La jerarquía del ID lo hace natural: muestrear co equivale a elegir la categoría
- Los modelos basados en item IDs no pueden hacer esto directamente: no hay una estructura desde la cual muestrear

Dos formas de convertir texto en códigos discretos

TIGER VS. VQ-REC

Esquema de VQ-Rec (Hou et al., WWW 2023)



texto ⇒ código ⇒ representación

- Objetivo: recomendación secuencial **transferible** entre dominios, sin ítems ni IDs en común
- El texto solo aporta el código; la representación del ítem proviene de **code embeddings**, que se aprenden
- El scoring es un **dot product** contra todos los vectores de ítems (recomendador discriminativo clásico), no una secuencia generada

Esquema de VQ-Rec (Hou et al., WWW 2023)

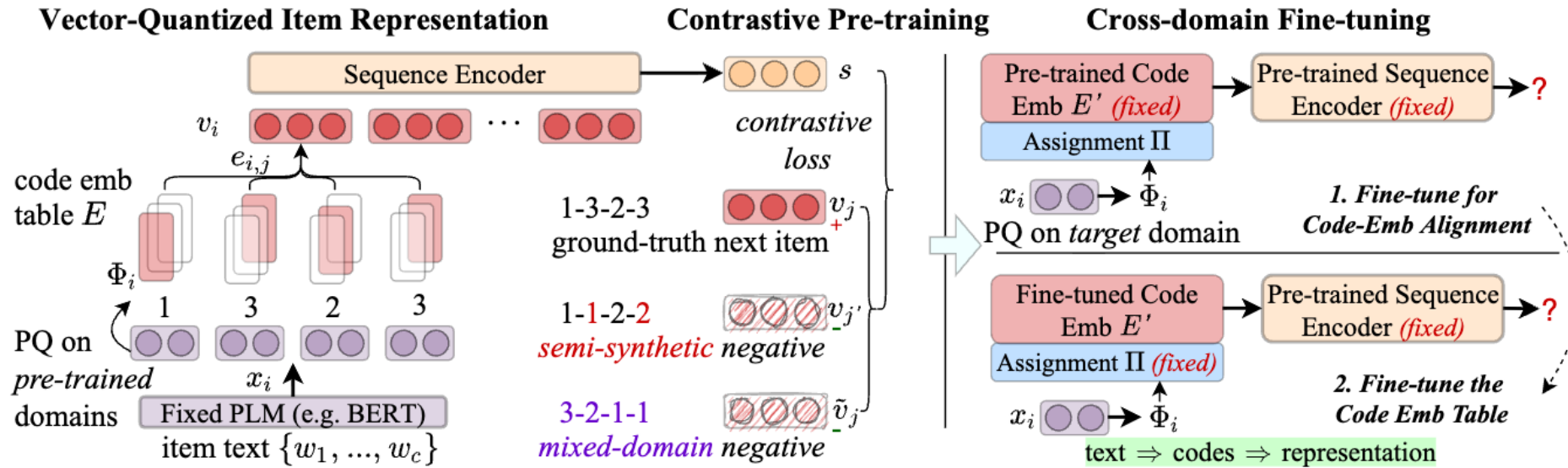


Figure 1: The overall framework of the proposed vector-quantized code-based transferable sequential recommender VQ-Rec.

- Objetivo: recomendación secuencial **transferible** entre dominios, sin ítems ni IDs en común
- El texto solo aporta el código; la representación del ítem proviene de **code embeddings**, que se aprenden
- El scoring es un **dot product** contra todos los vectores de ítems (recomendador discriminativo clásico), no una secuencia generada

Mismo ingrediente, distinto diseño

TIGER (Semantic IDs)

- Texto $\rightarrow$ Sentence-T5 $\rightarrow$ **RQ-VAE**
- Niveles **residuales** y jerárquicos: coarse $\rightarrow$ fine
- Cuantizador **aprendido** (reconstrucción + commitment)
- + un token extra para colisiones
- Código = un **identificador** para generar

VQ-Rec (códigos como índices)

- Texto $\rightarrow$ BERT $\rightarrow$ **(O)PQ** product quantization
- Los dígitos cuantizan **subvectores separados**: sin jerarquía
- La cuantización **no se entrena** junto con el recomendador
- Varios ítems pueden compartir todos sus códigos
- Código = un **índice a embedding tables**

Qué ocurre después de los códigos

- **TIGER:** el decoder Transformer **genera** los códigos del siguiente ítem token por token (generative retrieval); una lookup table mapea códigos $\rightarrow$ ítem
- **VQ-Rec:** los códigos de un ítem se convierten en embeddings y se promedian (**mean pooling**) en un solo vector; un Transformer codifica el historial; los ítems se **ordenan por dot product**
- TIGER no necesita item embedding table ni índice ANN; VQ-Rec igualmente puntúa todos los ítems
- Ninguno reemplaza directamente al otro: responden preguntas distintas (nuevo paradigma de retrieval vs. transferibilidad)

VQ-Rec: del texto a la representación del ítem

El text encoder entrega un vector semántico para el ítem i

$$\mathbf{x}_i = \text{BERT}([\text{CLS}; w_1; \dots; w_c])$$

Product quantization: cada dígito selecciona un código de su propio conjunto de centroides

$$c_{i,k} = \arg \min_j \|\mathbf{x}_{i,k} - \mathbf{a}_{k,j}\|_2, \quad \mathbf{x}_i = [\mathbf{x}_{i,1}; \dots; \mathbf{x}_{i,D}]$$

Vector del ítem = promedio de sus code embeddings (tablas aprendidas)

$$\mathbf{v}_i = \text{Pool}(\mathbf{e}_{1, c_{i,1}}; \dots; \mathbf{e}_{D, c_{i,D}})$$

Un Transformer codifica el historial; los scores se obtienen con un dot product contra todos los vectores de ítems

$$\mathbf{f}_j^0 = \mathbf{v}_{i_j} + \mathbf{p}_j, \quad \mathbf{F}^{l+1} = \text{FFN}(\text{Attn}(\mathbf{F}^l))$$

VQ-Rec: entrenamiento y transferencia

Contrastive loss sobre todos los ítems, con negativos mejorados (semi-sintéticos y de dominios mezclados)

$$\ell = -\frac{1}{B} \sum_j \log \frac{\exp(\mathbf{s}_j \cdot \mathbf{v}_j / \tau)}{\exp(\mathbf{s}_j \cdot \tilde{\mathbf{v}}_j / \tau) + \sum_{j'} \exp(\mathbf{s}_j \cdot \mathbf{v}_{j'} / \tau)}$$

Fine-tuning en un dominio nuevo: **solo se actualizan las tablas de code embeddings**; el Transformer se mantiene fijo (más un alineamiento por permutación de los códigos)

$$\hat{\mathbf{E}}^{(k)} = \Pi_k \mathbf{E}^{(k)}, \quad P(i_{t+1} \mid i_1, \dots, i_t) = \text{softmax}(\hat{\mathbf{s}} \cdot \hat{\mathbf{v}}_{i_{t+1}})$$

TIGER vs. VQ-Rec: resumen

	TIGER	VQ-Rec
Objetivo principal	Generative retrieval	Recomendación secuencial transferible
Códigos de ítems	Semantic IDs vía RQ-VAE (jerárquicos)	Códigos de product quantization (dígitos independientes)
Uso del código	Tokens para generar	Índices a embedding tables
Representación del ítem	Ninguna: el ítem es su ID	Promedio de code embeddings
Predicción	Decodificación autorregresiva + lookup	Dot product con todos los ítems
Cold-start / transferencia	Los ítems nuevos obtienen IDs desde su contenido	Dominios nuevos: fine-tuning de los code embeddings

Fuentes: Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023. Hou, He, McAuley, Zhao. Learning Vector-Quantized Item Representation for Transferable Sequential Recommenders. WWW 2023 (arXiv:2210.12316).

Limitaciones y trabajos posteriores

- IDs inválidos: el decoder puede generar códigos que ningún ítem tiene → decodificación restringida (trie)
- El cuantizador se entrena por separado del recomendador; sus errores no se propagan hacia atrás
- Benchmarks públicos pequeños; la escala de producción exige más cuidado (colisiones, ítems nuevos, actualización de IDs)
- Trabajos posteriores: señales más ricas del ítem en el ID, entrenamiento conjunto, despliegues industriales

Mensajes clave

- **Semantic IDs** = tuplas cortas de codewords derivadas del contenido del ítem: ítems similares comparten prefijos
- **RQ-VAE** los construye mediante residual quantization; un 4.º token resuelve las colisiones
- **TIGER** trata la recomendación como generación de secuencias: decodifica el Semantic ID del siguiente ítem
- Beneficios: mejor precisión, ítems cold-start, control de la diversidad
- **VQ-Rec** también cuantiza texto, pero para indexar embeddings con fines de transferencia, no para generar

Preguntas para discusión

1. ¿Por qué un código residual (coarse $\rightarrow$ fine) ayuda más que dígitos independientes?
2. ¿Qué se pierde si el cuantizador se congela antes de entrenar el recomendador?
3. ¿Cómo se podrían construir Semantic IDs para ítems con imágenes o audio?
4. ¿En qué casos convendría preferir el escenario de transferencia de VQ-Rec por sobre TIGER?

Referencias

- Rajput, S. et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.
- Hou, Y., He, Z., McAuley, J., Zhao, W. X. Learning Vector-Quantized Item Representation for Transferable Sequential Recommenders. WWW 2023. arXiv:2210.12316
- Zeghidour, N. et al. SoundStream (residual vector quantization). 2021.
- Jégou, H. et al. Product quantization for nearest neighbor search. 2011.